0423_기획 회의 정리

채팅서비스 기획

- 채팅은 기본적으로 인증된(로그인한 사용자)가 게임중이 아닐때 보내기 가능
- 받기의 경우 친구의 메세지만
- 보내는 사람의 입장에서 받는 사람의 상태는 중요하지 않음

게임서비스 기획

- 게임의 경우 게스트도 입장 가능

- 게임 규칙 : 제한 시간 1분, 3점, 승, 패, 동점 3가지 결과(빡종은 패배), gg 불가. 매칭할때 큐의 인덱스에 따라 왼쪽 혹은 오른쪽 배정(사용자 입장에서는 랜덤) 같은 상대랑 다시 붙는 기능은 없음. 화살표 + ws둘 다 받음.

- 게임 입장까지의 로직

- pong화면에서 매치잡기(버튼 눌러서 큐에 들어감) > 매치가 잡히면 안내창(상대방의 정보는 나오지 않음) > 동의하면 시작

게스트 기획

- 랜딩페이지에 “체험하기”버튼 추가
- 게스트는 채팅, 프로필, 친구추가 기능 사용 못함
- 게스트의 닉네임은 랜덤 생성.

- 생각해 볼 내용
- 게스트 히스토리? (로그인해서 5판을 한 경우 히스토리는?)
- 현재 모든 서비스에 인증된 사용자만 가능하도록 해둔 상태임. 게스트인 경우 확인하는 로직을 넣어야 하나? 아니면 그냥 빈 프론트 페이지를 만들어서 게스트는 거기로만 입장하게 해야하나?

서비스간 통식 방식

데이터의 형식에 따라 다른 통식 방식을 사용하는것이 적절할것 같습니다.

- Auth <-> User → REST(이미 하고 있음)
- 게임과 채팅 모두 게이트웨이에서 인증 한번 거치고 그 다음 시작.
- Game → WebSocket(실시간에 좋음)
- Chat → WebSocket(실시간에 좋음)
- Game → User(히스토리용) → REST or Redis + Pub/Sub
- 상태(Presence) → Redis + Pub/Sub
- (“상태”를 하나 혹은 다른 서비스에서 확인해야 할 경우 이 방식이 좋아 보임. 이미 redis 사용하고 있기에 도입도 어렵지 않음. 지금 유저 상태를 알아야 하는 서비스는 게임(매칭을 위해), 유저(표시를 위해) 2개이고 상태 표시를 위해 필요한 서비스는 인증(로그인할 때) 게임(시작할때 종료할때) 2개일 것으로 예상.

1. 전체 구조 개요

- 모든 요청의 진입점은 API Gateway
- Gateway는 인증과 라우팅만 담당
- 인증 후 각 서비스로 userId만 전달
- 실시간 통신(WebSocket)은 서비스와 클라이언트가 직접 연결
- 상태 정보는 Redis + Pub/Sub으로 관리

2. 진입 흐름

1) Client → Gateway (REST / WS 요청)
2) Gateway가 JWT 검증 및 userId 추출
3) 서비스 라우팅
4) 각 서비스로 userId 전달

3. 실시간(WebSocket) 흐름

1) Client → Gateway 핸드셰이크 (/ws/game, /ws/chat)
2) Gateway: JWT 검증 + userId 추출
3) 서비스로 연결 요청 (userId 전달)
4) Gateway는 연결 승인 후 연결 종료
5) 이후 Client ↔ Service 직접 양방향 통신

4. 상태 관리 (Presence)

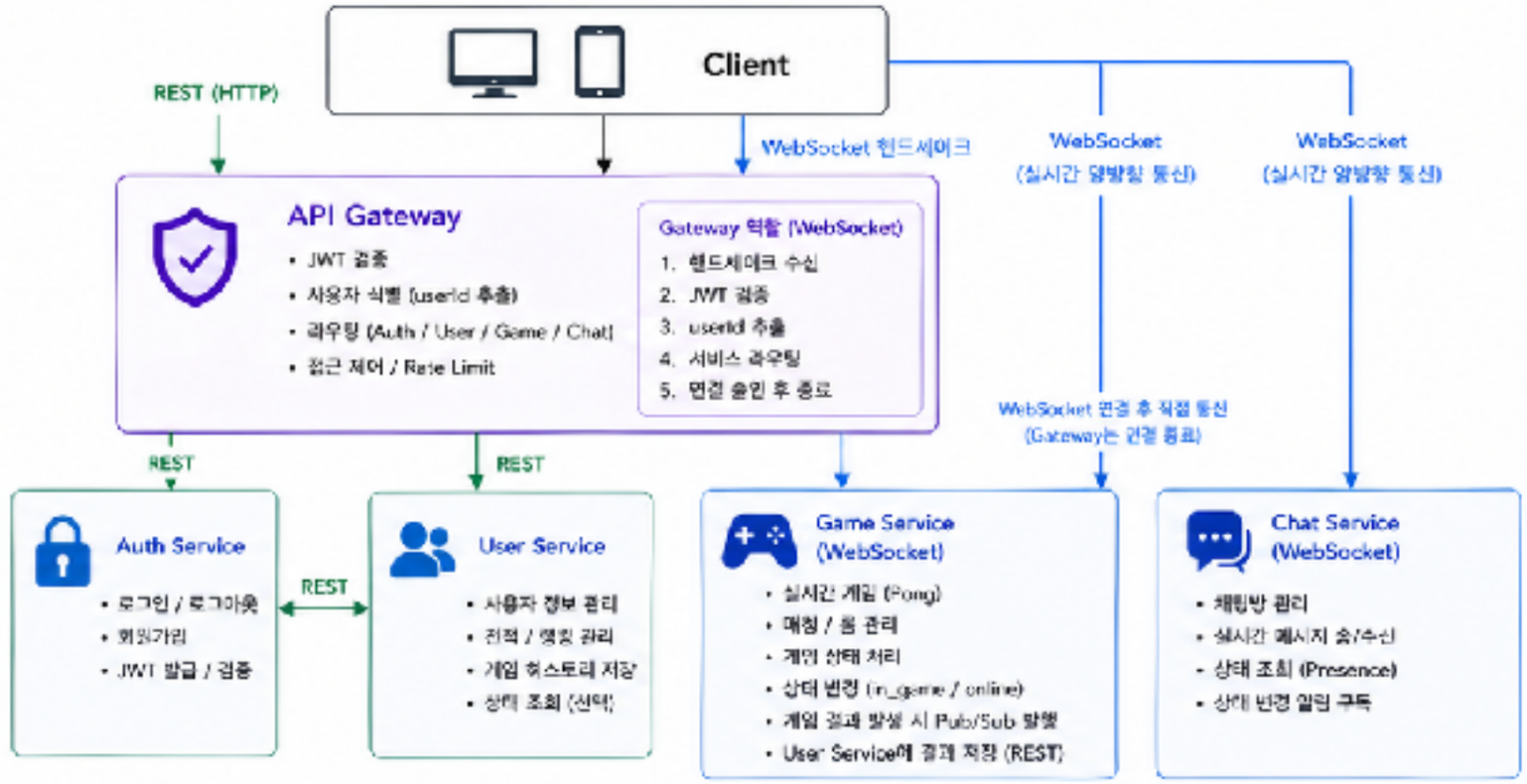
- Redis Key-Value에 사용자 상태 저장 (TTL)
  - Key: user:{userId}
  - Value: online | in_game | offline
  - TTL: 기본 30초 (Heartbeat 갱신)
- 상태 변경 시 Redis Pub/Sub으로 이벤트 발행

5. 상태 변경 트리거

- 로그인 (Gateway) → online
- 게임 시작 (Game) → in_game
- 게임 종료 (Game) → online
- 로그아웃 (Gateway) → offline
- 연결 해제 / 끊김 (Gateway/Service) → offline
- TTL 만료 → offline

6. 데이터 일관성 / 결과 저장

- Game 종료 시 Redis Pub/Sub으로 이벤트 발행
- User Service가 구독해서 전적 저장
- 실패 시 REST Fallback으로 결과 저장



전체 흐름 요약

1. 로그인 (REST) — Client → Gateway → Auth → JWT 발급
→
2. 요청 (REST) — Client → Gateway → 서비스 라우팅
→
3. WebSocket 핸드셰이크 — Client → Gateway (JWT 검증)
→
4. 연결 라우팅 — Gateway → 서비스 (userId 전달 후 종료)
→
5. 실시간 통신 — Client ↔ Service (직접 양방향)
→
6. 상태 변경 발생 — 서비스 → Redis 저장 + Pub/Sub 발행
→
7. 상태 변경 전파 — 구독 서비스들이 이벤트 수신 후 반영
→
8. 게임 종료 시 — Game → Pub/Sub → User Service 저장

범례

- REST (HTTP)
- WebSocket (초기 연결)
- WebSocket (직접 통신)
- Redis Pub/Sub (이벤트)
- REST (결과 저장 등)
- 상태 조회 (선택)

설명

- Gateway는 인증과 라우팅만 담당하고 이후 실시간 통신은 각 서비스에 직접 처리
- 모든 서비스는 Stateless 하며 userId 기반으로 동작
- 상태 정보는 Redis에 저장하고 Pub/Sub으로 변경을 전파
- Game 결과는 Pub/Sub 우선, 실패 시 REST로 User Service에 저장 (Fallback)
- 확장성: Gateway / 서비스 각각 독립 확장 가능

WebSocket 연결 요약

- /ws/game → Game Service로 연결
- /ws/chat → Chat Service로 연결
- Gateway는 핸드셰이크 후 연결에서 빠짐
- 이후 통신: Client ↔ Service 직접

상태 조회 방법

- 실시간 필요: Redis Pub/Sub 구독
- 단발 조회: 각 서비스가 Redis Key 조회